NoSQL

not only SQL (L0)

What is noSQL?

- https://www.mongodb.com/resources/basics/databases/nosql-explained

- NoSQL databases (also known as "not only SQL") store data differently than relational tables.
- NoSQL databases come in a variety of types based on their data model. The main types are:
  - Key-value databases
  - **Document databases**
  - Wide-column databases
  - Graph databases
- They provide flexible schemas and scale easily with large amounts of big data and high user loads.

3.3	**Databases and Data Management** *Understand, create and use SQL and NoSQL databases, as well as understand techniques to protect the privacy and integrity of data.*
Ref.	**Learning Outcome**
3.3.1	Determine the attributes of a database: table, record and field.
3.3.2	Explain the purpose of and use primary, secondary, composite and foreign keys in tables.
3.3.3	Explain with examples, the concept of data redundancy and data dependency.
3.3.4	Reduce data redundancy to third normal form (3NF).
3.3.5	Draw entity-relationship (ER) diagrams to show the relationship between tables.
3.3.6	Understand how NoSQL database management system addresses the shortcomings of relational database management system (SQL).
3.3.7	Explain the applications of SQL and NoSQL.
3.3.8	Write SQL statements and use a programming language to work with SQL and NoSQL databases.
3.3.9	Understand the need for privacy and integrity of data.
3.3.10	Describe methods to protect data.
3.3.11	Explain the difference between backup and archive.
3.3.12	Describe the need for version control and naming convention.
3.3.13	Explain how data in Singapore is protected under the Personal Data Protection Act to govern the collection, use and disclosure of personal data.

Shortcomings of RDBMS

- Scalability
  - Relational databases typically **scale vertically** (adding more powerful hardware), which can be expensive and have limits.
- Flexibility
  - Relational databases require a **fixed schema**, which makes it challenging to handle rapidly changing data models.
- Big Data Handling
  - Big data - extremely large and diverse collections of structured, unstructured, and semi-structured data that continues to grow exponentially over time.
  - Relational databases struggle with very large datasets, especially if the data does not fit well into a tabular format.

Shortcomings of RDBMS

- Semi-Structured and Unstructured Data Handling
  - Relational databases are designed primarily for structured data that fits into predefined tables and columns.

NoSQL Solution

- Scalability
  - Relational databases typically **scale vertically** (adding more powerful hardware), which can be expensive and have limits.
  - **Most** NoSQL databases are designed for horizontal scaling, allowing the database to distribute data across multiple servers. This makes it easier and more cost-effective to handle large-scale data.

* A "horizontally scalable" system is one that can increase capacity by adding more computers to the system.

NoSQL Solution

- Flexibility
  - Relational databases require a **fixed schema**, which makes it challenging to handle rapidly changing data models.
  - NoSQL databases support schema-less or flexible schemas, allowing for dynamic and heterogeneous data structures. This is particularly useful for applications that need to evolve quickly.

NoSQL Solution

- Big Data Handling
  - Big data - extremely large and diverse collections of structured, unstructured, and semi-structured data that continues to grow exponentially over time.
  - Relational databases struggle with storing and querying very large datasets.
  - NoSQL databases distributed file storage system are well-suited for managing large datasets and efficiently handles queries on distributed data

NoSQL Solution

- Semi-Structured and Unstructured Data Handling
  - Relational databases are designed primarily for structured data that fits into predefined tables and columns.
  - NoSQL databases can handle semi-structured and unstructured data. Eg. MongoDB excels at storing unstructured and semi-structured data due to its document-based model. JSON-like documents can represent complex hierarchical relationships directly within the document itself.

Applications of RDBMS

- Relational databases are ideal for applications requiring structured data, consistency, and complex querying.

- Example of applications/systems
  - Financial Systems eg. Banking,Payment gateway
    - Strong ACID compliance ensures accurate transaction processing

  - E-Commerce and Point of Sale (POS) Systems eg. Amazon, Shopify, retail chain POS systems
    - Manages product catalogs, order processing, and payment systems with transactional integrity.

Applications of NoSQL

- NoSQL databases excel in handling unstructured or semi-structured data, scaling horizontally, and supporting real-time or high-volume applications.
- Example of applications
  - Big Data Analytics
    - Handles massive datasets efficiently and optimized for distributed data storage and real-time processing.
  - Content Management Systems
    - Stores diverse content types (text, images, videos) without requiring rigid schemas.
  - E-Commerce Platforms
    - Flexible schema supports dynamic product catalogs and handles massive traffic spikes during sales or events.

MongoDB Server Installation

MongoDB Community Server version 3.4.9. **(Please DO NOT install other versions)**

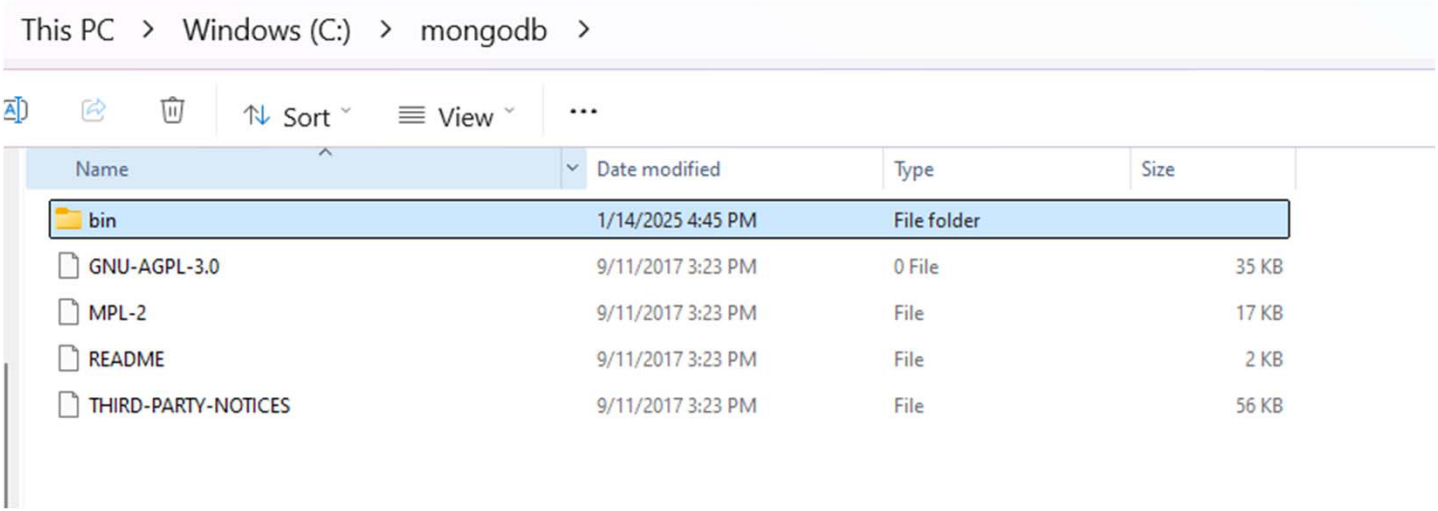
https://www.mongodb.com/try/download/community-edition/releases/archive

Download version 3.4.9 (windows_x86_64x64)

Unzip the zip file and copy the whole folder **mongodb-win32-x86_64-3.4.9** to your C: drive

Rename the folder to **mongodb**

MongoDB Server Installation

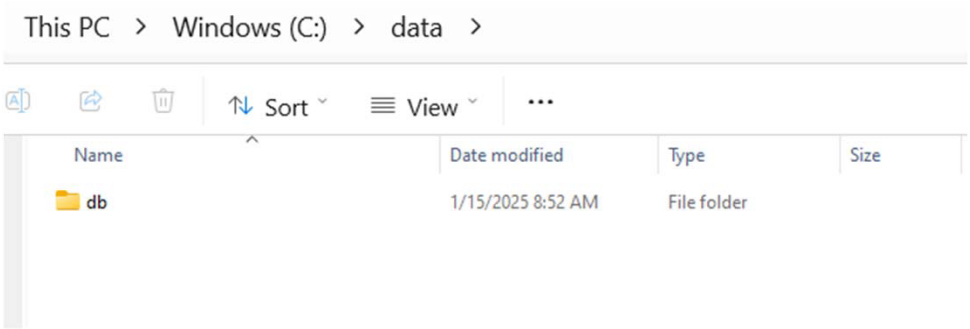


MongoDB Server Installation

(Next 2 steps are important - create the default folder where the database files are stored)

Create a new folder in your C: drive and name the folder: **data**

Create a new folder in the **data** folder and name the folder : **db**

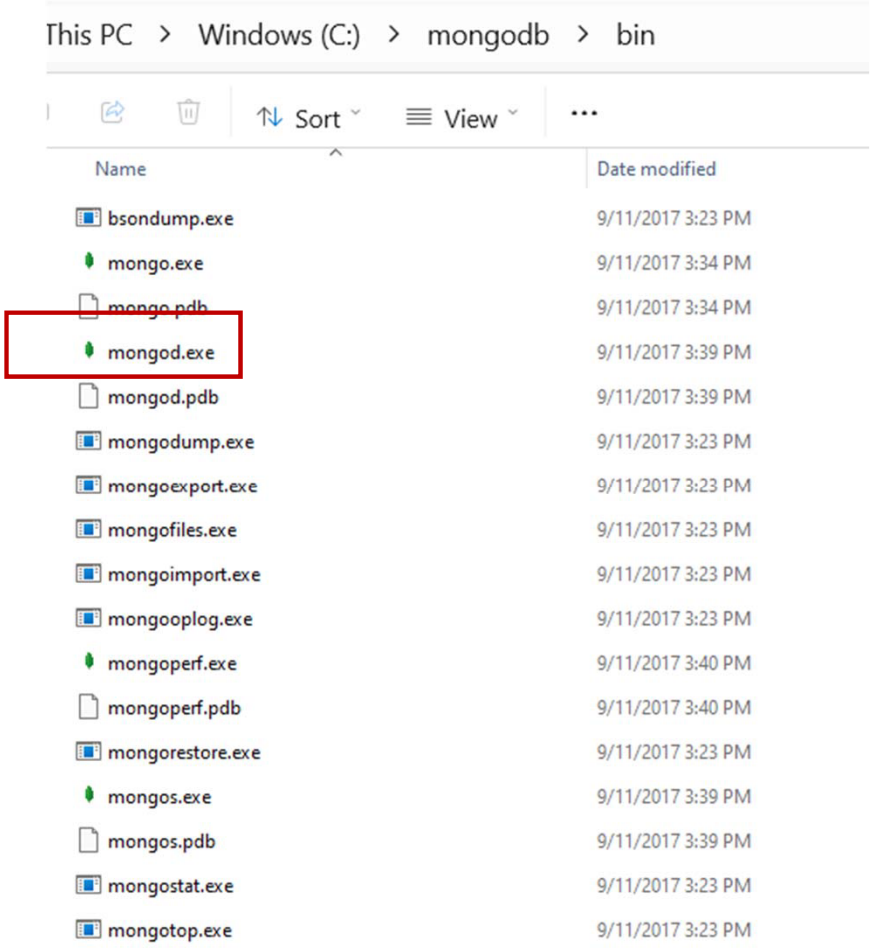


MongoDB Server Installation

To test that your installation is OK,

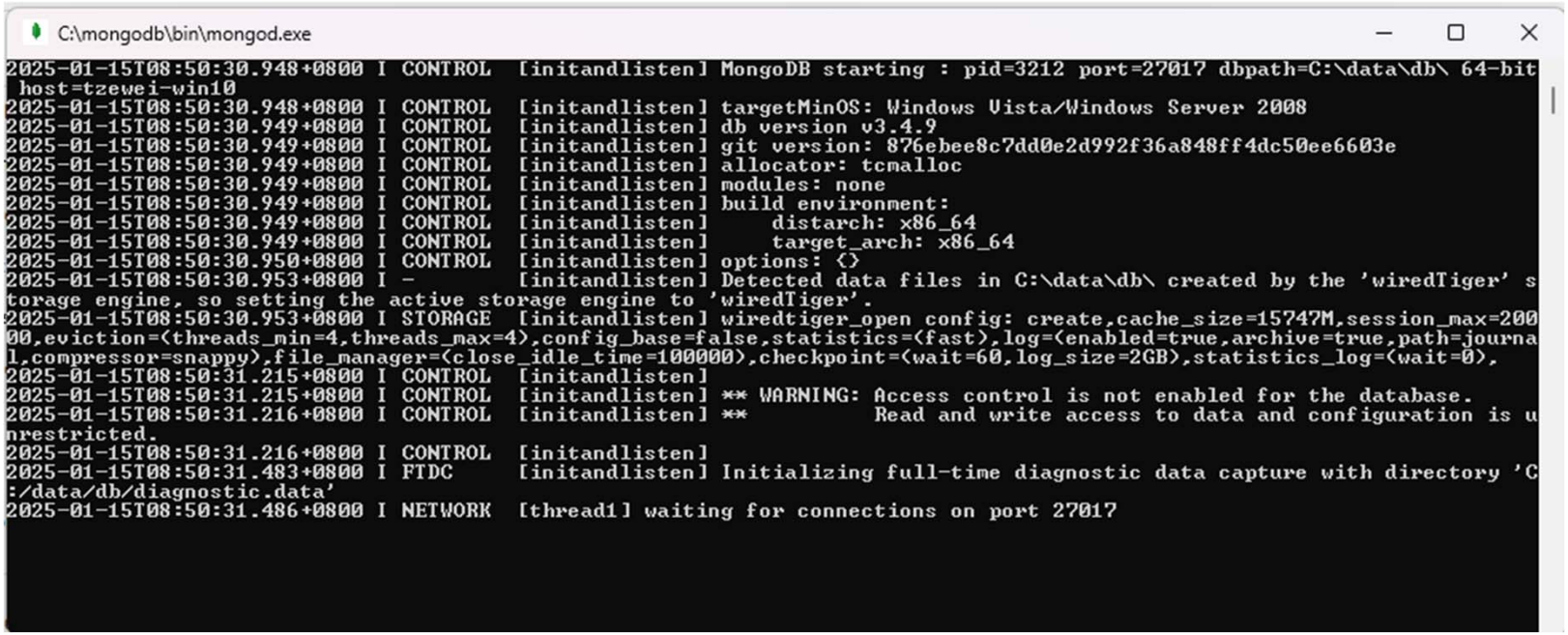
run **mongod.exe**

in **C:/mongodb/bin** folder



MongoDB Server Installation

You should see a "Windows console" with the last sentence that show: **waiting for connection on port 27017**



Pymongo Installation

Pymongo 3.13.0 - **make sure it works within your Jupyter Notebook**

https://pymongo.readthedocs.io/en/3.13.0/installation.html

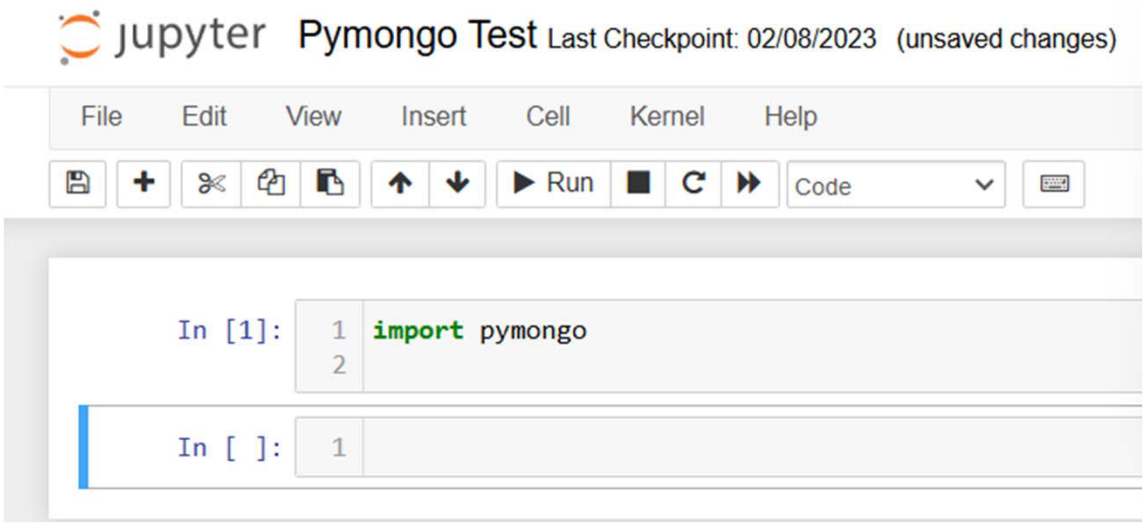
python -m pip install **pymongo==3.13.0**

Installing Pymongo in jupyter notebook

- Using pip : !pip install pymongo==3.13.0
- For anaconda : open anaconda prompt, type conda install -c anaconda pymongo

Pymongo Installation

Test that the installation works in your Jupyter Notebook. In a new cell, type **import pymongo** and run. If there is no errors shown, you are good.



Terms used in MongoDB (vs Relational Database)

MongoDB	SQL
Database	Database
Collection	Table
Document	Row
Field	Column

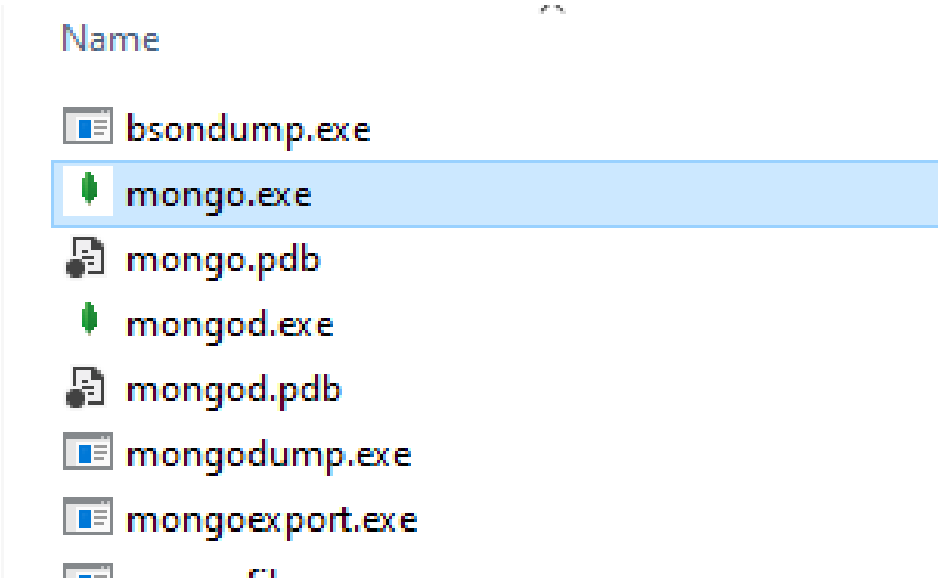
What is a Document Database?

What is a Document Database?

A document database (also known as a document-oriented database or a document store) is a database that stores information in documents.

```
{
  "_id": "5cf0029caff5056591b0ce7d",
  "firstname": "Jane",
  "lastname": "Wu",
  "address": {
    "street": "1 Circle Rd",
    "city": "Los Angeles",
    "state": "CA",
    "zip": "90404"
  }
  "hobbies": ["surfing", "coding"]
}
```

Starting Mongo Shell



Starting Mongo Shell

C:\mongodb\bin\mongo.exe

```
MongoDB shell version v3.4.24
connecting to: mongodb://127.0.0.1:27017
MongoDB server version: 3.4.24
Server has startup warnings:
2024-01-09T07:59:39.005+0800 I CONTROL  [initandlisten]
2024-01-09T07:59:39.005+0800 I CONTROL  [initandlisten] ** WARNING: Access control is not enabled for the database.
2024-01-09T07:59:39.006+0800 I CONTROL  [initandlisten] **          Read and write access to data and configuration is u
nrestricted.
2024-01-09T07:59:39.006+0800 I CONTROL  [initandlisten]
>
```

Mongo Shell

- Mongo shell is an interactive JavaScript shell which help you to manage a MongoDB server.
- It provides a set of commands to create, update, delete and query your MongoDB database.
- Note that in the exam environment, you will not have access to Mongo Shell.

Start/Stop MongoDB service

- Run MongoDB server ➔ \bin\mongod.exe
- Run Mongo Shell ➔ \bin\mongo.exe

Mongo Shell Command

- **help** – show help
- **show dbs** – show database names
- **use <db_name>** – set current database
- **show collections** – show collections in current database
- **exit/quit()** – quit the mongo shell
- **db.*collection*.drop()** - drop collection **replace collection with the collection name*
- **db.dropDatabase()** - drop database

Mongo Shell

- Start mongo shell

```
C:\mongodb\bin>mongo
MongoDB shell version v3.4.9
connecting to: mongodb://127.0.0.1:27017
MongoDB server version: 3.4.9
Server has startup warnings:
2026-01-13T08:49:07.948+0800 I CONTROL  [initandlisten]
2026-01-13T08:49:07.949+0800 I CONTROL  [initandlisten] ** WARNING: Access co
2026-01-13T08:49:07.949+0800 I CONTROL  [initandlisten] **          Read and
nrestricted.
2026-01-13T08:49:07.950+0800 I CONTROL  [initandlisten]
```

- Type the following statement in the shell:

```
> show dbs
admin  0.000GB
local  0.000GB
>
```

Mongo Shell

Insert data

```
> show dbs
admin  0.000GB
local  0.000GB
> use testdb
switched to db testdb
> db.person.insert({"name":"John Lim", "class":"25S6A", "hobbies":["running","kayaking"]})
WriteResult({ "nInserted" : 1 })
>
```

use testdb

db.person.insert({“name”:”John Lim”, “class”:”25S6A”, “hobbies”:[“running”, “kayaking”]})

Mongo Shell

Insert data

```
> show dbs
admin  0.000GB
local  0.000GB
> use testdb
switched to db testdb
> db.person.insert({"name":"John Lim", "class":"25S6A", "hobbies":["running","kayaking"]})
WriteResult({ "nInserted" : 1 })
>
```

show dbs

```
> show dbs
admin   0.000GB
local   0.000GB
testdb  0.000GB
>
```

Database is created only when a document is created

show collections

```
> show collections
person
>
```

Collection is automatically created when a document is added.

Mongo Shell

- Retrieve data

```
> db.person.find({})
{ "_id" : ObjectId("6965c0b14fe86c19046afe84"), "name" : "John Lim", "class" : "25S6A", "hobbies" : [ "running", "kayaki
ng" ] }
>
```

Mongo Shell

- Retrieve data

```
> db.person.find({})
{ "_id" : ObjectId("6965c0b14fe86c19046afe84"), "name" : "John Lim", "class" : "25S6A", "hobbies" : [ "running", "kayaki
ng" ] }
>
```

```
> db.person.find({"class":"25S6A"})
{ "_id" : ObjectId("6965c0b14fe86c19046afe84"), "name" : "John Lim", "class" : "25S6A", "hobbies" : [ "running", "kayaki
ng" ] }
> db.person.find({"class":"25S6B"})
```

```
> db.person.find({}).pretty()
{
        "_id" : ObjectId("6965fee12914f94621f9f155"),
        "name" : "John Lim",
        "class" : "25S6A",
        "hobbies" : [
                "running",
                "kayaking"
        ]
}
```

Mongo Shell

- Drop Collection

```
> db.person.drop()
true
```

- Drop Database

```
> show dbs
admin   0.000GB
local   0.000GB
testdb  0.000GB
> use testdb
switched to db testdb
> db.dropDatabase()
{ "dropped" : "testdb", "ok" : 1 }
> show dbs
admin  0.000GB
local  0.000GB
```

- Quit

```
> quit()

C:\mongodb\bin>
```

Import Book Database

```
Directory of C:\mongodb\bin

[.]                 [..]                bsondump.exe        mongo.exe           mongo.pdb           mongod.exe
mongod.pdb          mongodump.exe       mongoexport.exe     mongofiles.exe      mongoimport.exe     mongooplog.exe
mongoperf.exe       mongoperf.pdb       mongorestore.exe    mongos.exe          mongos.pdb          mongostat.exe
mongotop.exe
```

- Start the mongoDB server
- Check that there are no existing **bookdb** database
- Remove any **bookdb** database if it exists
- Use *mongoimport* to import the books.json file into the mongoDB server.

mongoimport --db bookdb --collection book --file c:\data\books.json

```
C:\mongodb\bin>mongoimport --db bookdb --collection book --file c:\data\books.json
2026-01-13T16:24:27.816+0800    connected to: localhost
2026-01-13T16:24:27.998+0800    imported 431 documents
```